



**Ciências
ULisboa**

Privacy and Data Security Project

First Delivery Report of Secure P2P Messaging App

Henrique Alípio	64452
Denis Ungureanu	56307
Pedro Marques	64857

MEI & MSI in Computer Science and Engineering

Introduction

The programming language used for this project is Java, with Maven as the build tool, and the application was developed in Visual Studio and IntelliJ. The user interface was created with JavaFX, and SSL sockets were implemented to ensure secure communication between peers. Version control was managed through GitHub, facilitating collaboration among team members.

The project is structured as follows:

- **CentralServer** – a server that controls users information and generates the truststore.
- **SecureP2PMessagingApp** – the main class of the project where the interface is loaded.
- **P2PServer** – handles incoming messages from peers.
- **P2PClient** – allows users to send messages.

Additionally, four core entities are defined:

- **Conversation** – manages message exchanges between users.
- **Message** – represents individual messages within conversations.
- **Share** - represents a share of user's key.
- **User** – identifies each user within the app.

Features

The secure P2P messaging application offers the following features:

- **Direct Messaging:** Enables users to initiate peer-to-peer communication by entering the recipient's username. If that username exists, the central server provides the recipient's IP address and port number to the sender. The sender can then send messages directly to the recipient.
- **Conversation Management and History:** Tracks all messages exchanged in the Conversations tab, with timestamps for chronological ordering retrieved from the cloud.
- **Message Searching:** Allows users to efficiently search for specific messages across all their conversations retrieved from the cloud.
- **Long-Term Storage of Messages:** Ensures messages are securely stored for extended periods using a distributed cloud storage system across three providers: **AWS S3**, **Firebase**, and **Azure**. This design guarantees privacy and security by encrypting messages before storage and employing Shamir's Secret Sharing algorithm for the secret keys. Redundancy is maintained to ensure availability, even in the event of a failure in one of the cloud providers.

Security

Application security is implemented through SSL sockets, establishing encrypted channels that guard against interception and tampering. Key management is facilitated using individual keystores, generated by each peer, alongside a truststore, created exclusively by the central server and shared across all peers. The central server also generates its own keystore. Upon joining the network, each peer generates a certificate and imports it into the truststore, enabling secure handshakes between peers when exchanging messages.

The peer-to-peer communication leverages SSL sockets to ensure robust implementation of all primary security pillars:

- **Confidentiality:** Ensures communication remains private by encrypting messages.
- **Integrity:** Prevents unauthorized tampering or alteration of messages.
- **Authenticity:** Validates the identity of peers through certificate-based verification.

The protocol used for secure communication is **TLSv1.2**, which is the default due to its compatibility with the majority of systems and JDK versions. The handshake process utilizes the cipher suite **TLS_AES_256_GCM_SHA384** for encryption algorithms, ensuring robust encryption of handshake messages. For certificate verification, the protocol employs the signature algorithm **rsa_pss_rsae_sha256**, which provides a modern, secure approach to digital signatures. Additionally, the algorithm **SHA256withRSA**, provided by the *BouncyCastle API*, is utilized for signature generation and verification, reinforcing the integrity and authenticity of the exchanged certificates.

In addition to securing communication, the system ensures **availability** by leveraging three cloud storage providers for storing encrypted messages. All data stored in the clouds is encrypted with the user's random key, ensuring its confidentiality. The key itself is securely stored in the clouds but divided into three shares using the **Shamir's Secret Sharing algorithm**. These shares are distributed among the clouds such that the key can be reconstructed using any two of the three shares. This redundancy ensures that even if one cloud fails, the system can reconstruct the key and access encrypted messages, maintaining high availability and reliability of the messaging platform

Cloud Storage

The system employs a distributed cloud storage architecture across three providers: **AWS S3**, **Firebase**, and **Azure**. This multi-cloud approach ensures that data is always accessible, even in the event of service disruption or failure in one of the providers.

The program is designed to retrieve data from the primary storage provider, **AWS S3**. If AWS S3 returns an error or becomes unavailable, the system seamlessly switches to **Firebase** to retrieve the data. In cases where both AWS S3 and Firebase are inaccessible, the system falls back to **Microsoft Azure** as the final retrieval point. This hierarchy ensures continuous availability of user data and minimizes service interruptions.

Each cloud storage provider organizes user data into structured folders for enhanced security and usability. Specifically:

- **User Folder:** Each user has a dedicated folder, containing the following:
 - **Encrypted Conversations:** All conversations associated with the user are stored in an encrypted format.

- **Shares Folder:** This folder contains the a share of the user’s key.
- **Dictionary Folder:** This folder contains encrypted words from the user’s messages.

user_alice	dictionaries	+ Adicionar campo
user_denis	henrique-pedro	encryptedData: "fDmCL6avCB15AQEipKt9qYde4RsAxnV4Q_Zi8IPX8SbceLT
user_henrique	shares_share2	S3wqHL9QJya00_fNEghZB9AANSDD4FI07XsNlKd3IAghb1VY
user_pedro		f0MIOYPA-VrZa360116MJCdqSsuDE33aaV-
		9LcZq8JE10ApF9_eFuTLzzf4TKC_CR0IZYeIBHROA61A70vUNF
		hKcVZeeba0p0k4TT8o0RMUDCTHqTeeHhUe0

Figure 1: Cloud Storage Architecture

Interface

Upon initialization, the application prompts the user to enter their login credentials, specifically the username and port number (with the IP address defaulted to localhost). Following successful login, the main menu interface is displayed, presenting several navigation options in the form of buttons:

- **Send Direct Message:** Form for recipient details (username, message).
- **Conversations:** Displays ongoing conversations and chat history with timestamps.
- **Search:** Searches for the word written in the text field in the all user’s conversation history.
- **Exit:** Closes the application and terminates active connections.

Additional Information

The exception handling mechanism is not implemented in the interface (e.g., same username, same port number, etc.).

Conclusion

The project successfully implements a peer-to-peer messaging system, achieving a high level of decentralization. A central server is utilized solely for aggregating peer information, ensuring minimal central dependency. The platform incorporates SSL for secure communication, JavaFX for a responsive and user-friendly interface, and a well-structured application architecture to enable secure, direct, and organized messaging between users.

Cloud storage plays a crucial role in storing user conversation data and a portion of the key required to restore user sessions. However, it does not interfere with the peer-to-peer communication itself, preserving the decentralized nature of the system.

Regarding performance, the reconstruction or creation of the user’s key occurs at the start of the session. While this process introduces a brief delay, it is essential to handle it on the main thread to ensure the user can immediately access the app with their key ready.